



Конспект по теме

"Синхронизация горутин"

Время изучения: 10 мин

Автор: Матвей Ефимов, Разработчик учебных программ и эксперт в Go, Python, SQL

Содержание

Нажмите на название раздела, чтобы перейти к нему

1. [Для чего нужна синхронизация горутин](#)
2. [Проблемы, возникающие при отсутствии синхронизации](#)
3. [Взаимные блокировки \(deadlocks\)](#)
4. [Основные средства синхронизации горутин в Go](#)
5. [Сравнение Mutex и atomic](#)
6. [Итоги](#)

1. Для чего нужна синхронизация горутин



Горутины — это лёгкие потоки исполнения, с помощью которых в Go удобно реализовывать параллельную и конкурентную обработку задач.



Однако при всей их эффективности **горутины работают в общей памяти**. Это значит, что **они могут одновременно обращаться к одним и тем же переменным и объектам**. В этом и кроется основная проблема: **параллельный доступ без согласования может привести к ошибкам**.

Когда несколько горутин обращаются к общей переменной, и хотя бы одна из них её изменяет, возникает состояние гонки данных (**data race**). Это приводит к непредсказуемому поведению программы: часть операций может потеряться, а результат — отличаться от ожидаемого. Даже одна строка вроде

counter++ может стать источником ошибки, если её одновременно выполняют несколько горутин.

Синхронизация позволяет избежать таких ситуаций. Она обеспечивает согласованный доступ к общим данным, регулирует порядок выполнения операций и помогает:

- защититься от гонок данных;
- избежать взаимных блокировок (**deadlocks**);
- обеспечить корректность расчётов и устойчивость программы в продакшене.

Go предоставляет разработчику разные средства синхронизации — от простых мьютексов (**sync.Mutex**) до каналов (**chan**). Их правильное применение — ключ к безопасному и эффективному использованию конкурентности в языке Go.

2. Проблемы, возникающие при отсутствии синхронизации

1. Гонки данных (data races)



Гонка данных возникает, когда две или более горутин одновременно обращаются к одной и той же переменной, при этом хотя бы одна из них производит запись, а синхронизация отсутствует.

Пример

```
var counter int

func main() {
    for i := 0; i < 1000; i++ {
        go func() {
            counter++ // небезопасная операция
        }()
    }

    time.Sleep(time.Second)
    fmt.Println(counter)
}
```

Что происходит в примере:

Ожидаемое значение — 1000. На практике результат может быть любым. Почему? Потому что операция `counter++` не является атомарной: она включает чтение, модификацию и запись. Если несколько горутин выполняют её одновременно, значения могут теряться.

Последствия:

- Потеря данных
- Непредсказуемое поведение
- Возможный сбой приложения в продакшене

Диагностика: Go предоставляет инструмент `go run -race`, который помогает выявлять гонки данных.

3. Взаимные блокировки (deadlocks)

Взаимная блокировка возникает, когда две или более горутины заблокированы в ожидании ресурсов, уже захваченных друг другом. В этом случае выполнение останавливается, и горутины не могут продолжить работу.



Взаимная блокировка возникает, когда две или более горутины заблокированы в ожидании ресурсов, уже захваченных друг другом. В этом случае выполнение останавливается, и горутины не могут продолжить работу.

Пример

```
var mu1, mu2 sync.Mutex

func main() {
    go func() {
        mu1.Lock()
        defer mu1.Unlock()

        time.Sleep(100 * time.Millisecond)
        mu2.Lock()
        defer mu2.Unlock()
    }()
}
```

```

go func() {
    mu2.Lock()
    defer mu2.Unlock()

    mu1.Lock()
    defer mu1.Unlock()
}()

time.Sleep(time.Second)
}

```

Что происходит:

Первая горутина захватывает `mu1` и ждёт `mu2`. Вторая — захватывает `mu2` и ждёт `mu1`. Обе горутины оказываются в состоянии взаимной блокировки.

Последствия:

- Зависание приложения
- Рост числа «застывших» горутин
- Отсутствие отклика на запросы

Диагностика: Часто выявляется по зависающим тестам или при ручной проверке логики блокировок.

3. Утечки горутин (goroutine leaks)



Утечка горутины возникает, когда она остаётся активной — продолжает выполнение или «висит» в ожидании события, которое никогда не произойдёт. Такая горутина не завершается и остаётся в памяти.

Пример

```

func worker(ch <-chan int) {
    for val := range ch {
        fmt.Println(val)
    }
}

func main() {
    ch := make(chan int)
    go worker(ch)
}

```

```
    // канал не закрывается, worker никогда не завершится
    time.Sleep(time.Second)
}
```

Что происходит:

Горутина **worker** ожидает данные из канала, но канал не закрывается и данные не приходят. В результате горутина «зависает» — остаётся в памяти и продолжает занимать ресурсы.

Последствия:

- Рост использования памяти
- Увеличение числа активных горутин
- Снижение стабильности приложения со временем

Признаки:

- Постоянный рост значения `runtime.NumGoroutine()`
- Замедление работы программы при длительном запуске

4. Голодание (starvation)



Голодание — это ситуация, при которой одна или несколько горутин систематически не получают доступ к ресурсу, поскольку другие горутин постоянно его захватывают.

Пример:

```
var mu sync.Mutex

func worker(id int) {
    for {
        mu.Lock()
        fmt.Println("Worker", id, "acquired lock")
        time.Sleep(100 * time.Millisecond)
        mu.Unlock()
        time.Sleep(10 * time.Millisecond)
    }
}
```

```
func main() {
    for i := 0; i < 5; i++ {
        go worker(i)
    }

    select {} // блокируем main навсегда
}
```

Что может произойти:

Если одна из горутин систематически быстрее остальных получает доступ к мьютексу (например, из-за особенностей планировщика), остальные могут «голодать» — им редко удаётся захватить ресурс.

Последствия:

- Снижение производительности
- Несправедливое распределение ресурсов
- Проблемы в системах, где важна балансировка нагрузки

4. Основные средства синхронизации горутин в Go

1. sync.WaitGroup

Назначение:

`WaitGroup` используется для ожидания завершения группы горутин. Это особенно полезно, если необходимо запустить несколько параллельных задач и продолжить выполнение программы только после их завершения.

Синтаксис:

```
var wg sync.WaitGroup
wg.Add(n)           // увеличить счётчик на n
wg.Done()           // уменьшить счётчик на 1 (вызвать внутри
горутин)
wg.Wait()           // блокирует, пока счётчик не станет 0
```

Пример:

```
func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Println("Worker", id, "started")
}
```

```

        time.Sleep(time.Second)
        fmt.Println("Worker", id, "finished")
    }

    func main() {
        var wg sync.WaitGroup

        for i := 1; i <= 3; i++ {
            wg.Add(1)
            go worker(i, &wg)
        }

        wg.Wait()
        fmt.Println("All workers completed")
    }

```

Пояснение:

- В `main` мы трижды вызываем `Add(1)`, чтобы увеличить счётчик.
- Каждая горутинa по завершении вызывает `Done()`, уменьшая счётчик.
- `wg.Wait()` блокирует выполнение, пока счётчик не станет равен нулю.

Типичные ошибки:

- Вызов `Add()` после `Wait()` может привести к зависанию.
- Пропущенный `Done()` внутри горутин — программа зависнет.
- Рекомендуется вызывать `defer wg.Done()` в начале горутин.

2. `sync.Mutex`

Назначение:

Mutex (мьютекс) обеспечивает взаимное исключение: только одна горутинa может получить доступ к критическому участку кода в каждый момент времени.

Синтаксис:

```

var mu sync.Mutex
mu.Lock()    // захват мьютекса
mu.Unlock()  // освобождение мьютекса

```

Пример:

```

var mu sync.Mutex
var counter int

```

```

func increment() {
    mu.Lock()
    counter++
    mu.Unlock()
}

func main() {
    for i := 0; i < 1000; i++ {
        go increment()
    }

    time.Sleep(time.Second)
    fmt.Println("Counter:", counter)
}

```

Пояснение:

Блок `Lock()` / `Unlock()` защищает операцию `counter++`, предотвращая гонки данных.

Антипаттерны:

- Пропущенный `Unlock()` приводит к взаимной блокировке.
- Рекомендуется использовать `defer mu.Unlock()` сразу после `Lock()` :

```

mu.Lock()
defer mu.Unlock()

```

3. sync.RWMutex

Назначение:

RWMutex — расширенная версия **Mutex**, позволяющая выполнять несколько операций чтения одновременно, но блокирующая доступ при записи. Полезно, когда чтения много, а записи — редкие.

Синтаксис:

```

var mu sync.RWMutex
mu.RLock()    // захват мьютекса на чтение
mu.RUnlock()  // освобождение мьютекса после чтения
mu.Lock()     // захват мьютекса на запись
mu.Unlock()   // освобождение мьютекса после записи

```


Пример:

```
type SafeMap struct {
    mu sync.RWMutex
    data map[string]string
}

func (s*SafeMap) Get(key string) string {
    s.mu.RLock()
    defer s.mu.RUnlock()
    return s.data[key]
}

func (s*SafeMap) Set(key, value string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.data[key] = value
}
```

Пояснение:

- Метод `Get()` может выполняться параллельно с другими `Get()`.
- Метод `Set()` блокирует доступ, чтобы избежать повреждения данных.

Рекомендации:

- Используйте **`RWMutex`**, только если прирост производительности измерен и оправдан.
- В остальных случаях проще и безопаснее применять обычный **`Mutex`**.

4. Пакет `sync/atomic`

Назначение:

Пакет `sync/atomic` предоставляет функции для атомарной работы с переменными — то есть операций, которые выполняются целиком и не прерываются. Это более лёгкий и быстрый способ синхронизации по сравнению с мьютексами, но применим только для простых типов: `int32`, `int64`, `uint32`, `uintptr`, `unsafe.Pointer`.

Примеры функций:

```
atomic.AddInt32(&counter, 1)
atomic.LoadInt32(&counter)
```

```
atomic.StoreInt32(&counter, 0)
```

Пример:

```
var counter int32

func increment() {
    atomic.AddInt32(&counter, 1)
}

func main() {
    for i := 0; i < 1000; i++ {
        go increment()
    }

    time.Sleep(time.Second)
    fmt.Println("Counter:", atomic.LoadInt32(&counter))
}
```

Пояснение: Атомарная операция **AddInt32** гарантирует корректное изменение значения даже при одновременном доступе из нескольких горутин.

5. Сравнение Mutex и atomic

Критерий	sync.Mutex	sync/atomic
Простота	Проще понять и использовать	Требует точного понимания атомарных операций
Гибкость	Позволяет синхронизировать любые данные	Работает только с примитивными типами
Производительность	Немного медленнее	Быстрее при высокой конкуренции
Безопасность	Меньше вероятность ошибок	Выше риск ошибок при сложной логике

6. Итоги

Теперь вы знаете четыре основных инструмента синхронизации в Go:

- **sync.WaitGroup** — для ожидания завершения группы горутин.
- **sync.Mutex** — для защиты критических секций кода.
- **sync.RWMutex** — для повышения производительности при множественном чтении.
- **sync/atomic** — для безопасной работы с примитивными переменными без использования блокировок.

Каждый из этих инструментов подходит для своих задач. Выбор зависит от контекста: требуемой производительности, читаемости кода и допустимой сложности реализации.

Основные инструменты синхронизации в Go

Инструмент	Назначение	Синтаксис (основные методы)	Когда использовать	Особенности / подводные камни
<code>sync.WaitGroup</code>	Ожидание завершения группы горутин	<code>Add(n)</code> , <code>Done()</code> , <code>Wait()</code>	Нужно дождаться окончания всех горутин	Нельзя вызывать Add() после Wait()
<code>sync.Mutex</code>	Взаимное исключение при доступе к разделяемым данным	<code>Lock()</code> , <code>Unlock()</code>	Работа с общими переменными	Забытый Unlock() приводит к взаимной блокировке
<code>sync.RWMutex</code>	Одновременное чтение и эксклюзивная запись	<code>RLock()</code> , <code>RUnlock()</code> , <code>Lock()</code> , <code>Unlock()</code>	Часто читаемые, редко изменяемые данные	Не перепутать RUnlock() с Unlock() , иначе возможна паника
<code>sync/atomic</code>	Атомарные операции над примитивами	<code>AddInt32()</code> , <code>LoadInt32()</code> , <code>StoreInt32()</code> и др.	Счётчики, флаги, индексы	Не подходит для сложных структур; требует внимательного применения

Рекомендации по стилю и best practices

1. **Используйте `defer` для освобождения блокировок.** Это снижает риск забыть вызвать `Unlock()` или `Done()`:

```
mu.Lock()
defer mu.Unlock()
```

2. **Избегайте вложенных блокировок.** Захват одного `Mutex` внутри другого — частая причина взаимных блокировок.
3. **Ограничивайте критическую секцию.** Помещайте внутрь блокировки только необходимую часть логики.
4. **Не используйте `Mutex` для защиты `nil`-ссылок.** Убедитесь, что защищаемый объект инициализирован.
5. **Оценивайте необходимость синхронизации.** Иногда можно обойтись локальными переменными или каналами.

6. **Не злоупотребляйте `sync/atomic`.** Даже если он быстрее — в большинстве случаев приоритетны читаемость и безопасность.
7. **Оптимизируйте только после измерений.** Переход на `RWMutex` или `atomic` должен быть обоснован замером производительности.

Частые ошибки и антипаттерны

Ошибка	Последствие	Как избежать
Вызов <code>Add()</code> после <code>Wait()</code>	<code>Wait()</code> может завершиться раньше, чем стартуют горютины	Всегда вызывайте <code>Add()</code> до запуска горютин
Забытый <code>Unlock()</code>	Взаимная блокировка	Используйте <code>defer</code>
Использование <code>atomic</code> с непригодным типом	Компиляция не пройдёт или возникнет UB	Применяйте только к поддерживаемым примитивным типам
Доступ к <code>map</code> без синхронизации	Падение во время выполнения	Используйте <code>Mutex</code> или <code>sync.Map</code>
Преждевременное использование <code>RWMutex</code>	Усложнение логики, больше шансов на ошибку	Начинайте с <code>Mutex</code> , меняйте при необходимости

Примеры ситуаций и рекомендации

Задача	Что использовать	Почему
Ожидание завершения 10 горютин	<code>sync.WaitGroup</code>	Прост и надёжен в подобных случаях
Инкремент счётчика посещений	<code>sync/atomic</code>	Быстро и эффективно, без блокировок
Доступ к общему слайсу	<code>sync.Mutex</code>	Обеспечивает эксклюзивный доступ при записи
Частое чтение кэша, редкая запись	<code>sync.RWMutex</code>	Позволяет масштабировать чтение без блокировки